

TEE is not a Healer, TEE-Rex is

Sadegh Keshavarzi and Gregory Chockler
University of Surrey

Alexey Gotsman
IMDEA Software

Trusted Execution Environments (TEE)



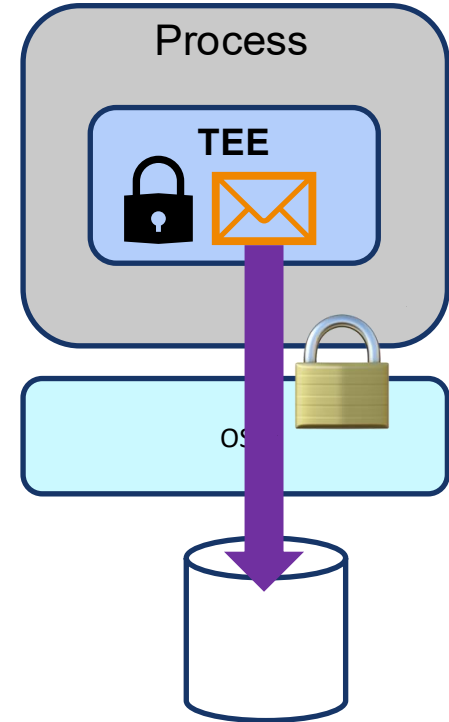
- Supports secure **code execution** and **volatile state storage**
- Reduces **Byzantine Fault Tolerance (BFT)** to **Crash Fault Tolerance (CFT)**



Data Persistence with TEEs

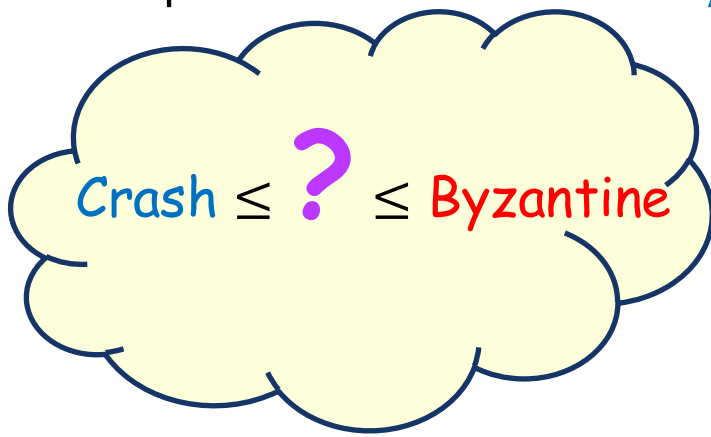
- External storage is **not** protected
- Durable state must be **sealed** (encrypted with TEE's secret key) before it can be persisted
 - Expensive

Is sealing secure?



Rollback Attacks

- On restart an attacker can replace the **latest** sealed version with an **older** one
 - **Undetectable**: both versions are correctly signed
- Restarted process behaves **correctly** afterwards



Our Contribution: Roadmap

Static Failure Models

Crash-Restart-Rollback (CRR)

Failures: Crash v Rollback

Dynamic Failure Models

Dynamic CRR (CRR-D)

Failures: Crash

Every process can rollback



Crash Fault
Tolerance
(CFT)



Tight Bounds for Registers
Resilience, Time Complexity
Persistent Storage Usage



Byz Fault
Tolerance
(BFT)

Our Contribution: Roadmap

Static Failure Models

Crash-Restart-Rollback (CRR)

Failures: Crash v Rollback

Dynamic Failure Models

Dynamic CRR (CRR-D)

Failures: Crash

Every process can rollback



Crash Fault
Tolerance
(CFT)



Tight Bounds for Registers
Resilience, Time Complexity
Persistent Storage Usage



Byz Fault
Tolerance
(BFT)

Static Failure Models: $CRR(k, r, b)$

Static Failure Models

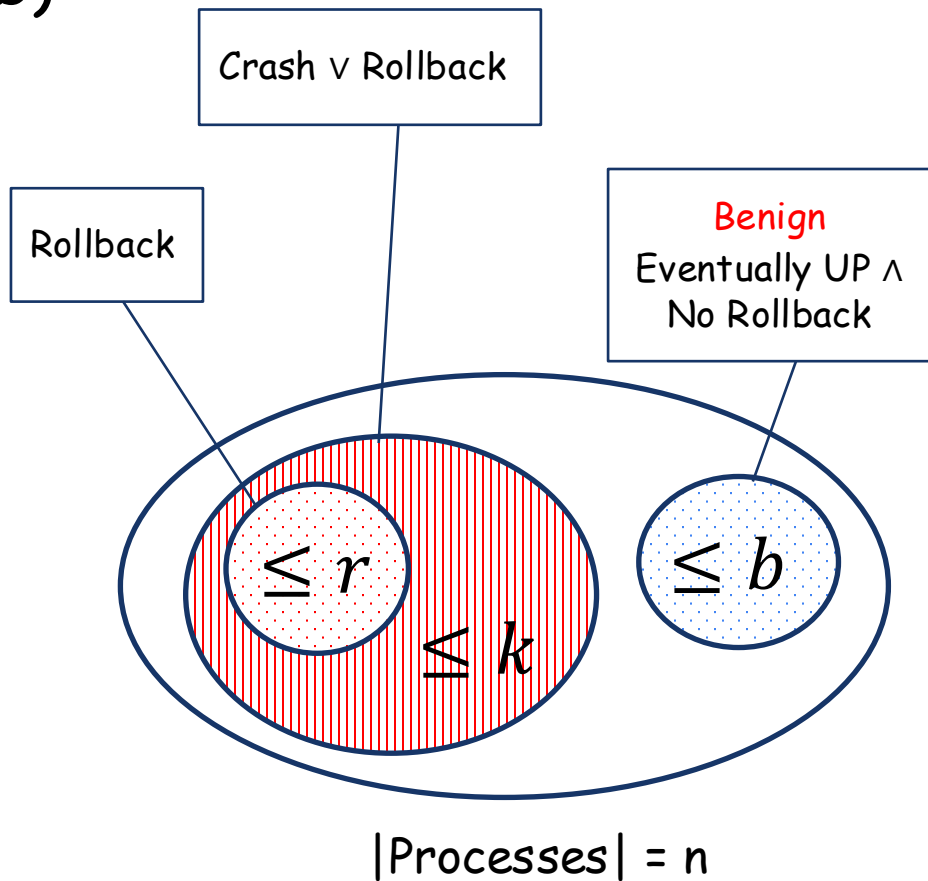
Crash-Restart-Rollback (CRR)

Failures: Crash $\vee$ Rollback

Tight Bounds for Registers

Resilience, Time Complexity

Persistent Storage Usage



Tight Bounds for $CRR(k, r, b)$

k : Crash v Rollback

r : Rollback

b : Benign

Wait-free atomic **MWMR** register can be implemented **IFF**

$$n \geq 2k + \min(r, b) + 1$$

	Resilience	Time Complexity	NV storage requirements
$r < b$	BFT	CFT	$2k + r + 1$
$r \geq b$	(BFT, CFT]	CFT	0

Our Contribution: Roadmap

Static Failure Models

Crash-Re

Fail

Crash

If all processes
can simultaneously
crash & restart, no
implementation is
possible

Dynamic Failure Models

Dynamic CRR (CRR-D)

Failures: Crash

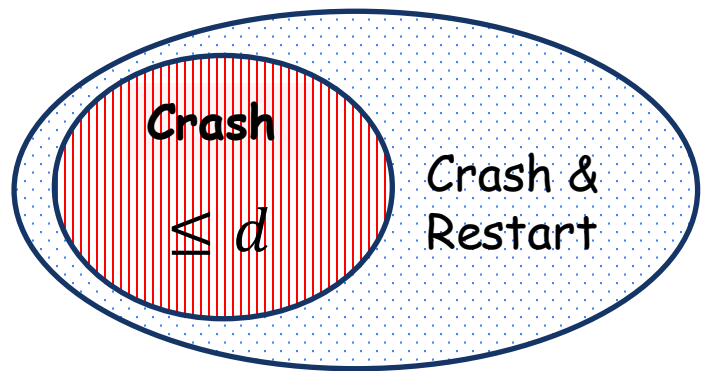
Every process can rollback

Right Bounds for Registers

Resilience, Time Complexity

Persistent Storage Usage

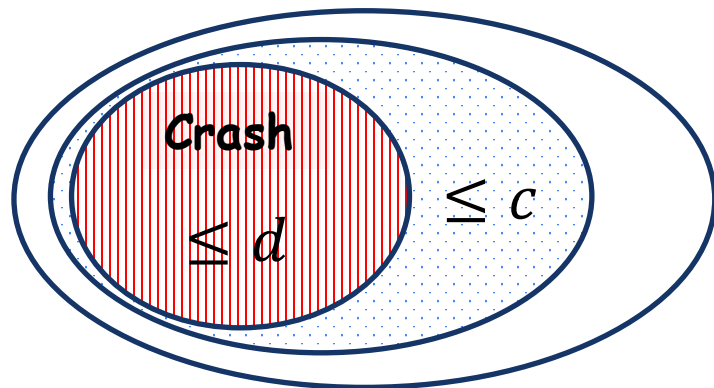
Dynamic Failure Models: CRR-D(d, c)



Good Case

Failures are separated by
 $\geq M \cdot \Delta$

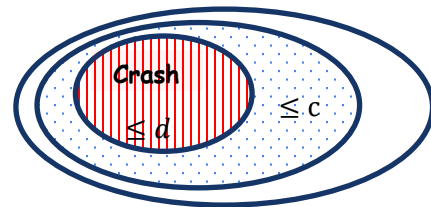
V



Bad Case

At most **$c \geq d$** can
crash & restart

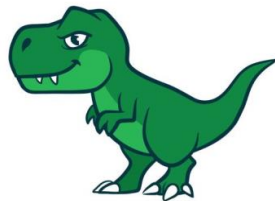
Tight Bounds for $CRR-D(d, c)$, $c \geq d$



Wait-free atomic **MWMMR** register can be implemented **IFF**

$$n \geq d + c + 1$$

Best Case Resilience	Good Case Latency	NV storage requirements
CFT	CFT	0



[KCG - DISC'25]

Upper Bounds for $CRR-D(d, c)$

TEE-Rex: New Wait-Free Atomic MWMR Register Construction

- Incorporates novel **distributed state recovery protocol** on restart to guarantee **termination** in the **good case** executions
- **First** correct implementation under **asynchrony** & **without** stable storage
- Uncovers **bugs** in 7 key prior publications
 - PODC, DISC, USENIX Security, CCS, NDSS, ...

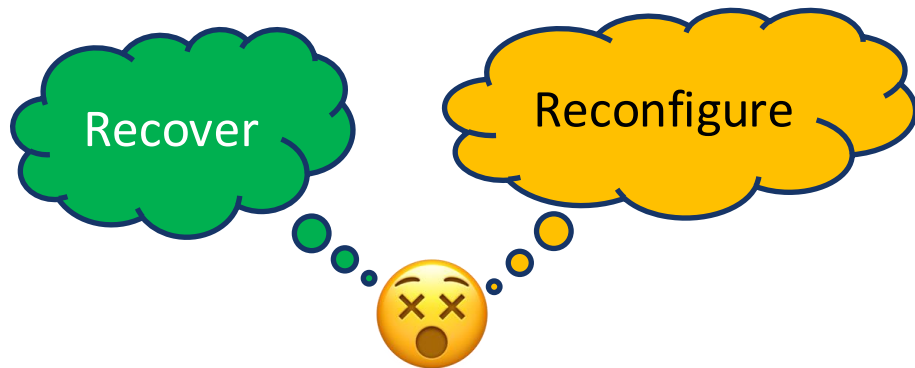
Ongoing work

- **Forking** resistance
 - Multiple enclaves in a process
 - Same code, **different** state
 - **Equivocation!**



Ongoing work (cont'd)

- Generalise the dynamic model
 - Protocol-independent benchmark



- Superior to reconfiguration?
 - Better resilience

Ongoing work (cont'd)

- Harder problems
 - Consensus
 - Partial synchrony
 - State-Machine Replication
 - Minimal Trusted Code Base

